Class activity – Encoder-Decoder Architectures

Name: Satyatma Chincholi

USN:1RVU23CSE414

Date: 9-3-2026

Encoder-decoder models represent a foundational architecture in deep learning, designed primarily for sequence-to-sequence (Seq2Seq) tasks. This framework allows a model to process an input sequence of one length and generate an output sequence of a different length, making it the backbone of modern machine translation, text summarization, and image captioning.

1. The Encoder Model

The encoder serves as the "understanding" phase of the architecture. Its primary role is to process the input data—such as a sentence in English or an image—and compress it into a high-dimensional numerical representation.

- **Mechanism:** In modern implementations like Transformers, the encoder utilizes a **self-attention mechanism**. This allows the model to weigh the importance of different parts of the input relative to each other (e.g., understanding which noun a pronoun refers to).
- **Output:** The encoder does not produce the final answer; instead, it outputs a set of hidden states or a final "Context Vector." This vector contains the semantic essence of the input, stripping away noise and retaining critical features.
- **Use Cases:** Encoder-only models (like BERT) are often used for classification, sentiment analysis, and named entity recognition where a complete "understanding" of the text is required.

2. The Decoder Model

The decoder acts as the "generative" phase. It takes the compressed information provided by the encoder and systematically unfolds it into the desired output format (e.g., a sentence in French).

- **Mechanism:** Decoders are typically **autoregressive**, meaning they generate output one token at a time. Each predicted word is fed back into the decoder to help predict the next word.
- **Masking:** To prevent the model from "cheating" during training, decoders use **masked self-attention**. This ensures that when the model is predicting a specific word, it can only look at the words that came before it, not the ones that follow.
- **Encoder-Decoder Attention:** The decoder also features a specialized attention layer that focuses back on the encoder's output. This allows the decoder to "look back" at specific parts of the original input while generating each part of the output.

3. The Context Vector (The Bottleneck)

The bridge between the encoder and decoder is the **Context Vector** (sometimes called the Latent Space). This is a fixed-size representation that acts as a bottleneck. It forces the model to learn the most efficient way to represent the input data. If the vector is too small, information is lost; if it is too large, the model may fail to generalize and instead just "copy" the input.

Python activity:

Untitled10.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

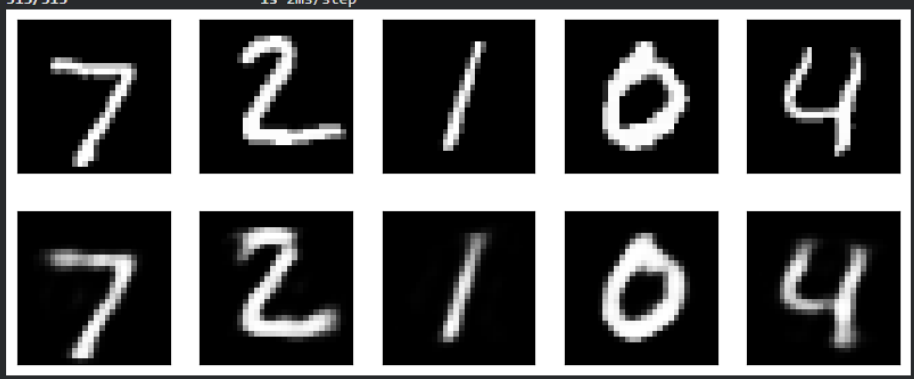
[1] 24s

```python
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist

# 1. Load and Prepare Data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))

# 2. Define Architecture
encoding_dim = 32  # This is the 'Context Vector' size (compression factor of 24.5)

# ENCODER
input_img = Input(shape=(784,))
encoded = Dense(encoding_dim, activation='relu')(input_img)

# DECODER
decoded = Dense(784, activation='sigmoid')(encoded)

# COMBINED MODEL (Autoencoder)
autoencoder = Model(input_img, decoded)

# 3. Compile and Train
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
autoencoder.fit(x_train, x_train,
                epochs=10,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test),
                verbose=1)
```

```python
# 4. Extract Encoder and Decoder for Testing
encoder = Model(input_img, encoded)
encoded_input = Input(shape=(encoding_dim,))
decoder_layer = autoencoder.layers[-1]
decoder = Model(encoded_input, decoder_layer(encoded_input))

# 5. Visualize Results
encoded_imgs = encoder.predict(x_test)
decoded_imgs = decoder.predict(encoded_imgs)

n = 5  # Number of digits to display
plt.figure(figsize=(10, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

    # Reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - loss: 0.3823 - val_loss: 0.1914
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1806 - val_loss: 0.1526
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1484 - val_loss: 0.1337
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1315 - val_loss: 0.1215
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1203 - val_loss: 0.1128
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1124 - val_loss: 0.1068
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.1068 - val_loss: 0.1027
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 0.1028 - val_loss: 0.0994
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.1001 - val_loss: 0.0971
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.0980 - val_loss: 0.0956
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



How this Activity Works:

1. **The Encoder:** Takes a 28×28 pixel image (784 values) and compresses it down to just **32 values**.
2. **The Context Vector:** The 32 values represent the "essence" of the digit.
3. **The Decoder:** Takes those 32 values and attempts to reconstruct the original 784-pixel image.
4. **Verification:** By looking at the output, you can see how much information the encoder was able to successfully capture within its bottleneck.